

Midterm Mastery

Contents

Step I — HW6 Results & Peer Discussion

Step II — What I Found Valuable So Far

Step III — Crashing & Recovering: Resilience Patterns Demo

HW6 Results — Finding the Bottleneck (Step I)

Setup: Go product search service on ECS Fargate (0.25 vCPU, 512 MB)

Users	RPS	Median Latency	CPU
5	~14.7	30ms	~28%
20	~58.8	29ms	~28%
200	~597	~20ms	~49%
500	Short burst	-	~85.6%

Bottleneck: CPU scaled linearly with load while memory stayed flat at ~8-10%.

CPU-bound operation → need more compute, not better code.

HW6 Results — Horizontal Scaling

Setup: ALB + Auto Scaling (min 2, max 4, target 70% CPU)

- **200 users through ALB:** ~600 RPS, lower per-instance CPU
- **Resilience test:** Stopped one instance during load → RPS dropped temporarily but recovered within ~1 minute. **0% failures.**
- **Takeaway:** Horizontal scaling provides both load distribution AND fault tolerance

HW6 Peer Discussion (piazza)

Commented on a classmate's gradual load increase approach (5→10→20→40→80 users):

- Their approach clearly showed auto scaling triggering at each step — CPU dropped from 79%→60% when the 3rd task launched
- **Key difference:** My 20 users only reached ~28% CPU vs their ~56%, likely due to different `wait_time` configurations
- Their insight about scale-out being faster than scale-in was something I didn't explore

What I Found Valuable — Key Themes (Step II)

1. **Thinking in Trade-offs**

- CAP theorem: can't have C + A + P simultaneously
- TCP vs UDP, strong vs eventual consistency, vertical vs horizontal scaling
- Every design decision is a trade-off

2. **The 8 Fallacies of Distributed Computing**

- "The network is not reliable, latency is not zero"
- Every assignment reinforced this — from HW1 timeouts to HW6 resilience testing

What I Found Valuable — Technical Growth

3. REST APIs & gRPC

- HW1/HW5: hands-on with Go REST APIs
- REST = human-readable, great for public APIs
- gRPC = Protocol Buffers + HTTP/2, faster for internal service-to-service

4. Concurrency & Race Conditions

- HW5: `sync.RWMutex` for concurrent reads, exclusive writes
- HW6: `sync.Map` for built-in thread safety
- Same problems appear at larger scale with replicated data

5. From Single Server to Distributed Systems

- HW1 (single EC2) → HW4 (MapReduce) → HW5 (load testing) → HW6 (ALB + Auto Scaling + ECS Fargate)

Step III — The Problem: Cascading Failure

Built a microservices e-commerce system in Go:

**Locust → API Gateway → Order Service → Payment Service (crashes!)
→ Inventory Service**

The scenario: Payment Service hangs for 30 seconds. What happens?

(screenshot: curl-normal-and-hang-test.png)

- Normal: **100ms** total latency
- With Payment hanging: **30.03 seconds** — entire system frozen

Phase 1 — No Resilience (Load Test)

(screenshots: *phase1-statistics.png*, *phase1-charts.png*)

Metric	Value
Median Latency	30,033 ms
Throughput	7.3 RPS
Total Requests (60s)	500

100 users all blocked waiting for Payment.

One failing service → entire system unusable.

The Fix — Three Resilience Patterns

(screenshots: *phase1-statistics.png*, *phase1-charts.png*)

Pattern	How It Works
Fail Fast	Background health checker pings services every 2s. If down → reject instantly (0ms)
Circuit Breaker	Track failures → after 3 fails, "open" circuit → block calls for 10s cooldown → probe recovery
Bulkhead	Isolate services into separate goroutine pools → Payment can't exhaust Inventory's resources

Inspired by Sam Newman's "Building Microservices"

Phase 2 — Fail Fast

(screenshots: *phase2-statistics.png*, *phase2-charts.png*)

Metric	Phase 1	Phase 2
Median Latency	30,033 ms	1 ms
Throughput	7.3 RPS	330.3 RPS
Total Requests	500	25,979

30,000x faster.

Requests fail, but fail instantly — resources freed immediately.

Phase 3 — Circuit Breaker

Scenario: Payment in flaky mode (50% random failure)

(screenshots: *phase3-statistics.png*, *phase3-charts.png*)

Metric	Value
Median Latency	32 ms
Throughput	7.3 RPS
Locust Failure Rate	0%

Circuit breaker returns HTTP 202 "pending" as graceful fallback.

System stays healthy under intermittent failures.

Phase 4 — Bulkhead + All Combined

All three patterns layered + parallel execution

(screenshots: phase4-statistics.png, phase4-charts.png)

Metric	Value
Median Latency	1ms
Throughput	332.1 RPS

Fail Fast catches down services.

Bulkhead isolates pools.

Circuit Breaker handles flaky failures.

Results Summary

Phase	Median	RPS	Key Insight
1 - No Resilience	30,033 ms	7.3	One failure → total collapse
2 - Fail Fast	1 ms	330.3	Instant rejection, 45x throughput
3 - Circuit Breaker	32 ms	295.7	Graceful degradation, 0% failures
4 - All Combined	1 ms	332.1	Full isolation + parallel calls

Latency: 30s → 1ms. Throughput: 7 → 332 RPS.

Key Takeaways

1. **Design for failure** — it's not "if" a service fails, it's "when"
2. **Fail fast** — don't waste resources waiting for dead services
3. **Circuit breakers** — allow graceful degradation and automatic recovery
4. **Bulkheads** — isolate failures so they don't cascade
5. **Measure everything** — without metrics, you can't prove your fix works